

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 6

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name : Deep Learning

Problem Statement :

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. Start.
2. Import required libraries.
3. Download and load the Apple Leaf Disease dataset.
4. Identify the disease classes and preprocess the images.
5. Resize images to 128×128 pixels and normalize pixel values.
6. Split the dataset into training and validation sets.
7. Create the CNN model with convolution and pooling layers.
8. Add flatten, dense, dropout, and Softmax output layers.
9. Compile the model using Adam optimizer and cross-entropy loss.
10. Train the model for selected epochs.
11. Evaluate the model and plot accuracy and loss graphs.
12. Stop.

Code :

```
import kagglehub
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import os
path = kagglehub.dataset_download(
    "mhantor/apple-leaf-diseases"
```

```

)
print("Path to dataset files:", path)
print("\nDataset contents:")
print(os.listdir(path))
BATCH_SIZE = 32
IMG_SIZE = (128, 128)

# Check folders inside downloaded dataset

folders = [
    f for f in os.listdir(path)
    if os.path.isdir(os.path.join(path, f))
]

print("\nFolders found:")
print(folders)
adjusted_path = path

if len(folders) == 1:

    possible_path = os.path.join(
        path,
        folders[0]
    )

    subfolders = [
        f for f in os.listdir(possible_path)
        if os.path.isdir(
            os.path.join(possible_path, f)
        )
    ]
    if len(subfolders) >= 2:
        adjusted_path = possible_path

print("\nUsing dataset path:")
print(adjusted_path)

# LOAD TRAINING DATA

train_ds = tf.keras.utils.image_dataset_from_directory(

    adjusted_path,

    validation_split=0.2,

    subset="training",

    seed=123,

    image_size=IMG_SIZE,

```

```

        batch_size=BATCH_SIZE
    )

# LOAD VALIDATION DATA
val_ds = tf.keras.utils.image_dataset_from_directory(
    adjusted_path,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

# GET CLASS NAMES
class_names = train_ds.class_names
print("\nDetected classes:")
print(class_names)
print(
    "\nNumber of classes:",
    len(class_names)
)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(
    buffer_size=AUTOTUNE
)
val_ds = val_ds.cache().prefetch(
    buffer_size=AUTOTUNE
)
num_classes = len(class_names)
model = models.Sequential([
    # Input
    layers.Input(
        shape=(128, 128, 3)
    ),
    # Normalize pixels
    layers.Rescaling(1./255),
    # CNN Layer 1
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu'
    ),
    layers.MaxPooling2D(
        (2, 2)
    ),
    # CNN Layer 2
    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),

```

```

layers.MaxPooling2D(
    (2, 2)
),
# CNN Layer 3
layers.Conv2D(
    128,
    (3, 3),
    activation='relu'
),
layers.MaxPooling2D(
    (2, 2)
),
# Flatten
layers.Flatten(),
# Fully connected layer
layers.Dense(
    128,
    activation='relu'
),
# Dropout
layers.Dropout(0.5),
# Output layer
layers.Dense(
    num_classes,
    activation='softmax'
)
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
EPOCHS = 10
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
plt.figure(figsize=(12, 5))

# Accuracy
plt.subplot(1, 2, 1)
plt.plot(
    acc,
    label='Train Accuracy'
)

```

```

plt.plot(
    val_acc,
    label='Validation Accuracy'
)
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1, 2, 2)
plt.plot(
    loss,
    label='Train Loss'
)
plt.plot(
    val_loss,
    label='Validation Loss'
)
plt.title('Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
# MODEL EVALUATION
import numpy as np
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# Get true labels and predictions
y_true = []
y_pred = []

for images, labels in val_ds:

    predictions = model.predict(images, verbose=0)

    predicted_classes = np.argmax(
        predictions,
        axis=1
    )

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_classes)

# CLASSIFICATION REPORT
print("\nClassification Report:\n")
print(

```

```
classification_report(  
    y_true,  
    y_pred,  
    target_names=class_names  
)  
)  
  
# CONFUSION MATRIX  
cm = confusion_matrix(  
    y_true,  
    y_pred  
)  
print("\nConfusion Matrix:\n")  
print(cm)  
  
# DISPLAY CONFUSION MATRIX  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=cm,  
    display_labels=class_names  
)  
disp.plot(  
    xticks_rotation=45  
)  
plt.title("Apple Leaf Disease Confusion Matrix")  
plt.show()
```

Output :

```
*** Downloading to /root/.cache/kagglehub/datasets/mhantor/apple-leaf-diseases/1.archive...
100%|██████████| 6.70M/6.70M [00:01<00:00, 4.91MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/mhantor/apple-leaf-diseases/versions/1

Dataset contents:
['Apple_cedar_rust', 'Apple_scab', 'Apple_black_rot']

Folders found:
['Apple_cedar_rust', 'Apple_scab', 'Apple_black_rot']

Using dataset path:
/root/.cache/kagglehub/datasets/mhantor/apple-leaf-diseases/versions/1
Found 480 files belonging to 3 classes.
Using 384 files for training.
Found 480 files belonging to 3 classes.
Using 96 files for validation.

Detected classes:
['Apple_black_rot', 'Apple_cedar_rust', 'Apple_scab']

Number of classes: 3
Model: "sequential_3"
```

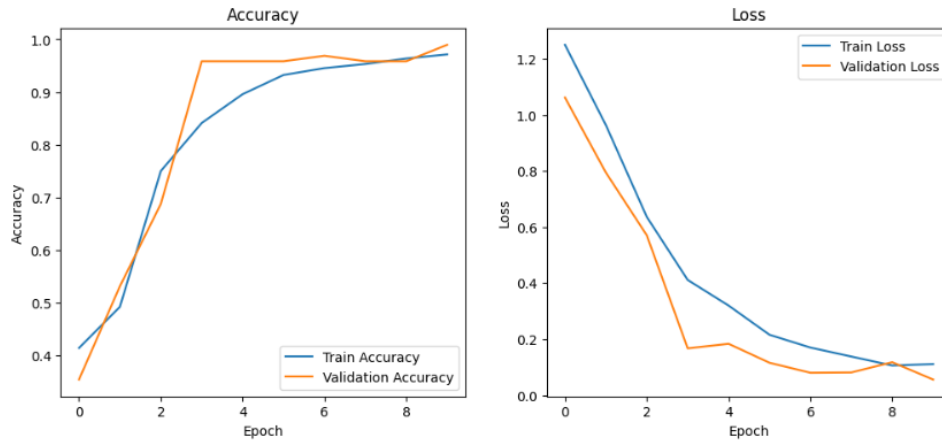
Layer (type)	Output Shape	Param #
rescaling_3 (Rescaling)	(None, 128, 128, 3)	0
conv2d_9 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_9 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_10 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_10 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_11 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_11 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_6 (Dense)	(None, 128)	3,211,392
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 3)	387

```
Total params: 3,305,027 (12.61 MB)
Trainable params: 3,305,027 (12.61 MB)
Non-trainable params: 0 (0.00 B)
```

```

...
Epoch 1/10
12/12 ----- 19s 1s/step - accuracy: 0.4141 - loss: 1.2507 - val_accuracy: 0.3542 - val_loss: 1.0624
Epoch 2/10
12/12 ----- 21s 1s/step - accuracy: 0.4922 - loss: 0.9640 - val_accuracy: 0.5312 - val_loss: 0.7946
Epoch 3/10
12/12 ----- 14s 1s/step - accuracy: 0.7500 - loss: 0.6364 - val_accuracy: 0.6875 - val_loss: 0.5706
Epoch 4/10
12/12 ----- 15s 1s/step - accuracy: 0.8411 - loss: 0.4112 - val_accuracy: 0.9583 - val_loss: 0.1671
Epoch 5/10
12/12 ----- 15s 1s/step - accuracy: 0.8958 - loss: 0.3198 - val_accuracy: 0.9583 - val_loss: 0.1833
Epoch 6/10
12/12 ----- 14s 1s/step - accuracy: 0.9323 - loss: 0.2156 - val_accuracy: 0.9583 - val_loss: 0.1156
Epoch 7/10
12/12 ----- 14s 1s/step - accuracy: 0.9453 - loss: 0.1699 - val_accuracy: 0.9688 - val_loss: 0.0799
Epoch 8/10
12/12 ----- 14s 1s/step - accuracy: 0.9531 - loss: 0.1379 - val_accuracy: 0.9583 - val_loss: 0.0810
Epoch 9/10
12/12 ----- 15s 1s/step - accuracy: 0.9635 - loss: 0.1059 - val_accuracy: 0.9583 - val_loss: 0.1176
Epoch 10/10
12/12 ----- 15s 1s/step - accuracy: 0.9714 - loss: 0.1110 - val_accuracy: 0.9896 - val_loss: 0.0557

```



Classification Report:

	precision	recall	f1-score	support
Apple_black_rot	0.97	1.00	0.99	34
Apple_cedar_rust	1.00	1.00	1.00	27
Apple_scab	1.00	0.97	0.99	35
accuracy			0.99	96
macro avg	0.99	0.99	0.99	96
weighted avg	0.99	0.99	0.99	96

Confusion Matrix:

```

[[34  0  0]
 [ 0 27  0]
 [ 1  0 34]]

```

